Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Allocation contiguë, indexation mémoire, dégradation d'array (Array Decay) et pointeurs

Synthèse exécutive : Ce cours aborde l'organisation interne et la manipulation des tableaux à une dimension en langage C. Il explicite la disposition contiguë des éléments en mémoire vive (stack) et le calcul d'adresse garantissant un accès direct en complexité O(1). Le cours détaille également le phénomène central d'Array Decay (conversion automatique de l'identifiant d'un tableau en pointeur vers son premier élément), démontre pourquoi une fonction ne peut pas déterminer la taille d'un tableau reçu et formalise l'obligation de transmettre la taille effective en paramètre explicite.

1. Définition, allocation contiguë et calcul d'adresse mémoire O(1)

En langage C, la déclaration d'un tableau statique (ex: int tab[5];) demande au compilateur de réserver une zone contiguë en mémoire. Pour 5 entiers de 4 octets chacun, 20 octets consécutifs sont alloués sur la pile. L'accès aux éléments via l'opérateur crochet tab[i] n'effectue aucun parcours séquentiel : le système applique directement la formule d'arithmétique système Adresse_Base + (i * sizeof(type)). Puisque le premier élément n'a aucun décalage par rapport au début du bloc, son indice est obligatoirement 0. Tenter d'accéder à l'indice N (ex: tab[5] pour une taille de 5) constitue un débordement de mémoire (Buffer Overflow) entraînant un comportement indéterminé.

- Tableau unidimensionnel : Structure de données statique et homogène stockant un nombre fixe d'éléments de même type.
- Contiguïté mémoire : Reservation d'un bloc ininterrompu de N * sizeof(type) octets en mémoire vive.
- Indexation de 0 à N-1 : L'indice i représente le décalage (offset) par rapport à l'adresse de base du tableau.
- Complexité constante O(1) : Calcul direct d'adresse via Adresse(tab[i]) = Adresse_Base + (i * sizeof(type)).

Cas concret enseignant : *Le professeur a utilisé l'analogie d'un meuble de rangement vertical comportant des tiroirs rigoureusement identiques et collés les uns au-dessus des autres, numérotés de 0 à N-1. Si l'on connaît l'emplacement du sol (adresse de base) et la hauteur exacte d'un tiroir, on calcule instantanément la hauteur du tiroir i sans avoir à mesurer ou ouvrir les tiroirs précédents.*

2. Pointeurs, passage par adresse et mécanisme d'Array Decay

Pour des raisons de performance, le langage C interdit la copie intégrale d'un tableau lors d'un appel de fonction (pas de passage par valeur). Lorsqu'un tableau est passé en paramètre, son nom se 'dégrade' (Array Decay) en un pointeur de type type* pointant vers la case &tab;[0]. Cela signifie que toute modification effectuée dans la fonction altère directement le tableau original de la fonction appelante. Cependant, le type 'tableau de N éléments' étant perdu dans le scope de la fonction, l'opérateur sizeof ne renvoie plus la taille totale du tableau mais la taille du pointeur (4 ou 8 octets). Il est donc strictement obligatoire d'inclure la taille du tableau dans la signature de la fonction.

- Array Decay : Conversion implicite du nom d'un tableau en un pointeur vers son premier élément (&tab;[0]) lors de son passage à une fonction.
- Passage par adresse : Transmission de l'emplacement mémoire original pour éviter une copie inutile et permettre la modification directe.
- Arithmétique des pointeurs : Équivalence stricte entre tab[i] et *(tab + i).
- Perte de taille : Impossibilité pour la fonction appelée de déterminer dynamiquement la taille du tableau transmis.

Cas concret enseignant : *L'enseignant a expliqué que transmettre un tableau à une fonction ne revient pas à fabriquer une copie exacte de sa maison dans le jardin de son ami (passage par valeur), mais à lui fournir les coordonnées GPS précises de sa maison (passage par adresse). L'ami se déplace aux coordonnées transmises et repeint directement les murs du bâtiment d'origine.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante-t-il ou affiche-t-il des valeurs absurdes lorsque j'essaie de lire ou d'écrire dans tab[5] alors que mon tableau a été déclaré avec une taille de 5 (int tab[5];) ?
Réponse du professeur : Pour un tableau déclaré de taille N = 5, les indices valides vont strictement de 0 à 4 (soit de 0 à N-1). L'indice 5 correspond à une 6ème case située en dehors du bloc mémoire alloué. Le langage C n'effectue aucun contrôle automatique des bornes à l'exécution : vous effectuez une lecture ou écriture hors limites (Buffer Overflow / Out-of-bounds access), ce qui provoque la lecture de valeurs 'poubelles' ou un crash du programme (Segmentation Fault).

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Non, c'est un piège classique ! Dans le corps de la fonction, en raison de l'Array Decay, 'tab' n'est plus perçu comme un tableau complet mais comme un simple pointeur (int*). L'expression sizeof(tab) renvoie donc la taille du pointeur en mémoire (4 ou 8 octets selon votre architecture) et non la taille globale du tableau. Le calcul donnera un résultat erroné. C'est la raison pour laquelle la taille effective du tableau doit obligatoirement être passée en paramètre distinct à la fonction.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C nommée 'remplir_et_calculer_moyenne' qui prend en paramètre un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur de saisir au clavier la valeur de chaque élément, puis calculer et retourner la moyenne arithmétique sous la forme d'un nombre à virgule (double). La fonction doit être sécurisée contre la division par zéro.

Résolution pas à pas :
1. Prototypes et types : Déclarer la fonction double remplir_et_calculer_moyenne(int tab[], int taille); 2. Sécurité : Ajouter une condition préalable pour vérifier si taille <= 0. Si vrai, retourner 0.0 pour éviter toute division par zéro. 3. Initialisation de l'accumulateur : Déclarer double somme = 0.0; pour accumuler la somme exacte sans conversion tronquée. 4. Parcours et saisie : Utiliser une boucle for avec un indice i variant de 0 à taille - 1. 5. Remplissage et cumul : Demander la saisie à l'utilisateur via scanf("%d", &tab;[i]); et ajouter directement la valeur à somme. 6. Calcul et retour : Diviser somme par taille et retourner la valeur de type double.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { if (taille <= 0) { return 0.0; } double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Saisir l'élément [%d] : ", i); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; } // Exemple d'exécution avec taille = 3, saisies : 12, 14, 16 -> Moyenne = 14.000000

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau d'entiers déclaré par int t[10]; ?

R : L'indice 9 (car les indices s'étendent de 0 à N-1, soit de 0 à 9 pour 10 éléments).

Q : Que contient l'identifiant d'un tableau lorsqu'il est employé seul sans crochets dans une expression ou un appel de fonction ?

R : L'adresse mémoire de son premier élément (&tab;[0]), suite au mécanisme d'Array Decay.

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure sa taille en paramètre ?

R : Car lors du passage à la fonction, le tableau est converti en un simple pointeur (Array Decay). La fonction perd l'information sur la dimension du tableau et ne peut plus déterminer sa taille via sizeof.